

Aula 4 — Aquisição analógica: o ADC e o LM35

O pino que não é booleano, e o meio grau que sobra

Microcontroladores — DENE/UFMT

OBJETIVOS

- Distinguir os três caminhos de um mesmo pino — o driver de saída, o buffer digital de entrada e a chave analógica de entrada — e explicar por que os dois caminhos de entrada não funcionam ao mesmo tempo.
- Descrever o mecanismo da conversão: capacitor de retenção, DAC de pesos binários, comparador, e a busca binária que o hardware executa.
- Derivar por que são exatamente dez decisões, e o que acontece quando uma delas sai errada.
- Justificar o tempo de aquisição a partir da impedância da fonte e da capacitância de retenção, e decidir se um divisor resistivo pode ser lido diretamente.
- Configurar o conversor campo a campo, em binário, e derivar o custo de uma conversão em ciclos de máquina.
- Converter o código de dez bits em décimos de grau usando aritmética inteira, e enunciar a resolução resultante em graus Celsius.

1 O número que ficou afirmado

O encontro 3 fechou com uma comparação: atualizar a tela custa cerca de 5 200 ciclos, e uma conversão analógica custa cerca de 60. O primeiro número foi derivado linha a linha da folha de dados do HD44780. O segundo foi afirmado.

Hoje ele é derivado. E, no caminho, uma coisa que o curso vinha assumindo desde o encontro 0 deixa de valer: a de que um pino é um bit.

2 O pino tem três caminhos

Até aqui, LATD0 = 1 acendeu um LED e PORTD devolveu o estado de uma linha. Nas duas operações o pino se comportou como uma célula de memória com um fio saindo. Ele não é isso.

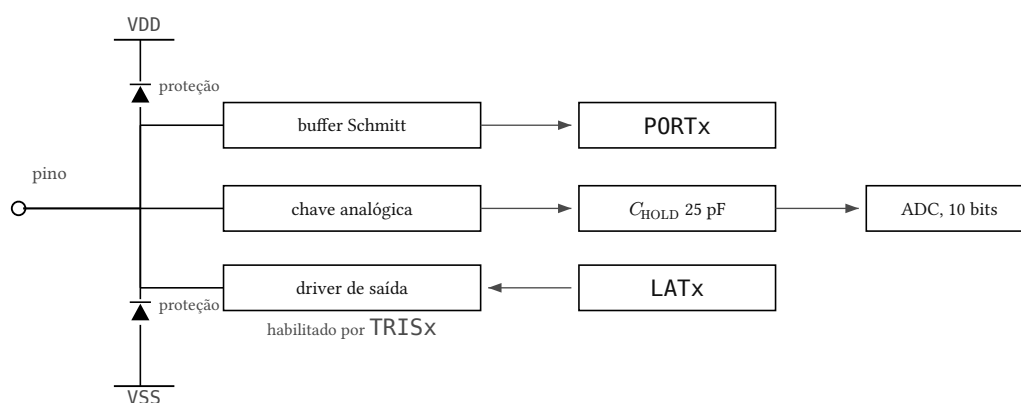


Figura 1: O mesmo pino, com três caminhos e dois diodos. O ADC não lê o pino: ele lê um capacitor que foi ligado ao pino por alguns microssegundos.

São três circuitos ligados ao mesmo pedaço de metal, e a configuração escolhe qual deles está ativo.

2.1 Saída: LATx e o driver

Escrever 1 em LATD0 não guarda um bit “no pino”. O que esse bit faz é comandar um par de transistores que liga o pino a VDD ou a VSS: o driver **impõe** ao pino a tensão

correspondente ao valor do registrador — 5 V para 1, 0 V para 0 — e fornece a corrente necessária para mantê-la.

Daí sai tudo o que o curso já usou sem justificar. A corrente máxima por pino existe porque o transistor tem tamanho finito. O LED precisa de resistor porque o driver mantém a tensão e não limita a corrente. E LAT antes de TRIS é uma consequência: enquanto TRIS não libera o driver, o valor de LAT não chega ao pino; depois que libera, chega imediatamente — inclusive o lixo que estava lá.

2.2 Entrada digital: PORTx e o buffer

Ler PORTD não mede tensão: compara. Um buffer confronta a tensão do pino com um limiar e devolve 0 ou 1. Qualquer valor entre 0 e 5 V é forçado a virar um dos dois — é o pino que **responde** uma pergunta binária, e não o programa que observa o mundo.

2.3 Entrada analógica: a chave e o capacitor

O terceiro caminho não compara nada. Ele fecha uma chave que liga o pino a um capacitor interno, deixa esse capacitor se carregar, e entrega a tensão retida ao conversor. É o único caminho que preserva **quanto**.

Os dois caminhos de entrada disputam o mesmo pino. Quando o canal analógico está habilitado, o buffer digital daquele pino é desligado, e aquele bit de PORTx passa a ser lido como 0. Não é falha: é a configuração pedindo uma coisa e o programa lendo outra.

CONCEITO

Por isso o padrão do curso é **declarar**, nunca herdar: depois do reset, vários pinos nascem analógicos, e a primeira linha do main desfaz isso.

```
ADCON1 = 0b00001111; // PCFG<3:0> = 1111: todos os canais em modo digital
```

Cada canal que você quiser como analógico é então habilitado explicitamente. Tudo digital, e o analógico se declara.

Os bits de configuração pertencem ao bootloader, e o `#pragma config` da sua aplicação é ignorado — **o código que veio antes do seu**, encontro 0. O estado dos pinos logo após o reset, portanto, não é negociável: só resta desfazê-lo em tempo de execução.

NESTE KIT

Os diodos de proteção não são decorativos. Eles conduzem se a tensão no pino passar de ~ 0,3 V acima de VDD ou abaixo de VSS. Aplicar 12 V num pino não queima o chip pela tensão: queima pela corrente que passa a circular por um diodo que não foi feito para isso. É a mesma razão pela qual a garra de terra do osciloscópio nunca vai nos pontos de 12 V da placa.

NOTA

O buffer digital também tem histerese — dois limiares, um para subir e outro para descer — e o que acontece entre eles é assunto do R5, onde se mede.

3 Como o conversor funciona

3.1 O ADC não lê o pino

Ele lê um capacitor. A chave analógica fecha, o capacitor C_{HOLD} — cerca de 25 pF — se carrega através da resistência do canal mais a resistência da fonte, e só então a chave abre e a conversão começa. O que é convertido é a tensão que sobrou no capacitor no instante da abertura.

Se o capacitor não teve tempo de chegar à tensão da fonte, o erro é de aquisição, e nenhuma média de amostras o corrige — ele é sistemático.

O tempo de aquisição cresce com a impedância da fonte. A folha de dados recomenda no máximo ~ 2,5 kΩ. Duas consequências:

CONCEITO

O LM35 pode ser lido diretamente: a impedância de saída dele é de fração de ohm.

Um divisor resistivo de $1\text{ M}\Omega$ não pode. Ele funciona no multímetro, cuja entrada tem dezenas de megaohms, e falha no ADC, que precisa **puxar** carga. A saída disso é baixar as resistências do divisor ou pôr um seguidor de tensão entre ele e o pino.

3.2 Bits viram volts

A tensão retida é comparada com a saída de um conversor digital-analógico. O DAC transforma bits em volts usando um arranjo de capacitores cujos valores estão organizados em potências de dois: o capacitor do bit mais significativo vale o dobro do seguinte, que vale o dobro do próximo. Ligar um bit é somar a contribuição do capacitor correspondente.

O comparador devolve nível alto quando a tensão retida é maior que a tensão proposta pelo DAC, e nível baixo quando é menor. Uma resposta, um bit de informação — e mais nada.

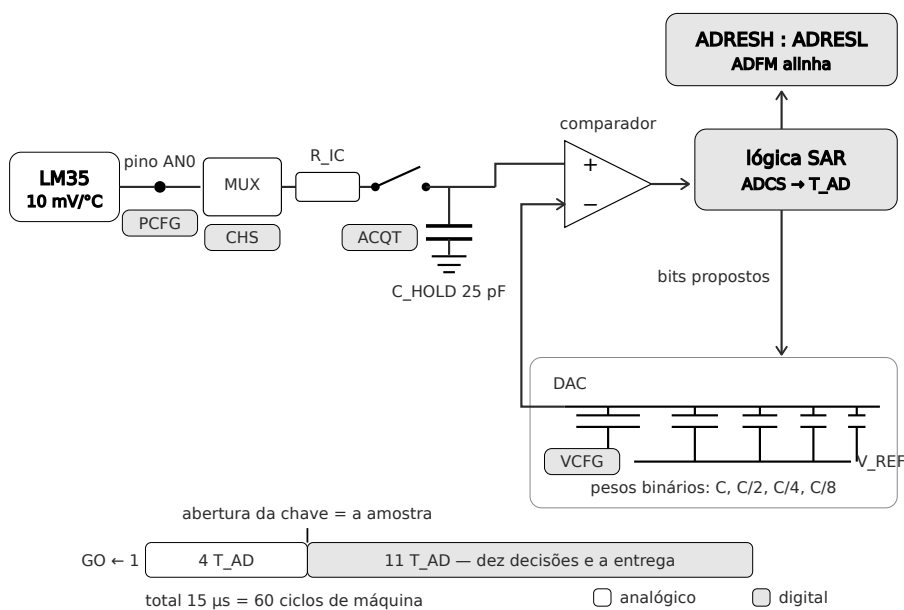


Figura 2: O circuito inteiro, com cada campo de configuração ancorado no bloco que ele comanda. Abaixo, a mesma conversão no eixo do tempo: a abertura da chave é o instante da amostra.

Uma simplificação útil. Na figura — como na folha de dados — o DAC e o C_{HOLD} aparecem como blocos separados. No silício, o arranjo de capacitores de pesos binários é o mesmo capacitor de retenção, e a comparação acontece por redistribuição de carga. Separar em dois blocos preserva o que importa aqui: o tempo, os dez passos e a régua de referência.

NOTA

3.3 Aproximações sucessivas

A conversão é uma busca binária. O conversor compara a tensão retida com metade da faixa, decide um bit, corta a faixa pela metade e repete.

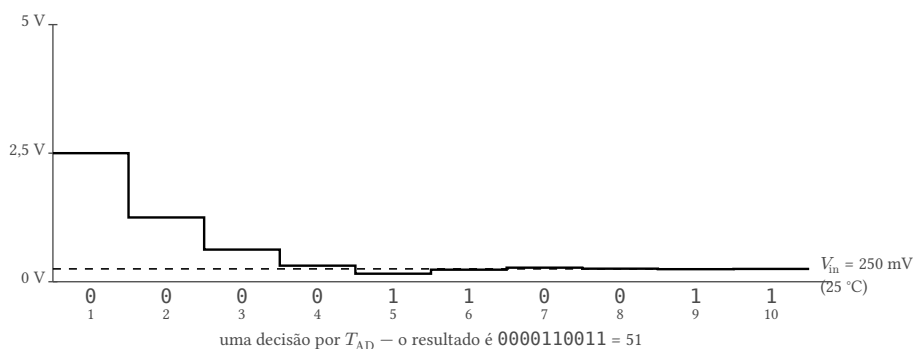


Figura 3: Dez decisões para dez bits, uma por T_{AD} . Os quatro primeiros bits são zero porque o LM35 a 25 °C usa 5% da faixa de entrada — a figura mostra o desperdício antes de o texto falar dele.

São dez decisões, e a folha de dados cobra **onze** T_{AD} pela conversão: o décimo primeiro é o tempo de descarregar o resultado e liberar o conversor.

3.3.1 Por que exatamente dez

O número dez não foi escolhido: ele é obrigatório, e a conta cabe em duas linhas.

O comparador é o único instrumento de medida do circuito, e ele só sabe responder **sim** ou **não** — uma resposta binária carrega, no máximo, um bit de informação. Distinguir entre 1024 códigos exige 10 bits de informação. Logo, nenhuma estratégia consegue converter em menos de $\log_2 1024 = 10$ comparações.

Dividir ao meio é a estratégia que atinge esse limite, e atinge por um motivo preciso: só a divisão ao meio torna as duas respostas igualmente prováveis, e é aí que a resposta carrega um bit inteiro. Qualquer outro ponto de corte torna uma das respostas mais provável, e uma resposta esperada informa menos. Perguntar “a tensão é maior que 4,9 V?” quase sempre recebe **não** — a pergunta foi desperdiçada.

O invariante. Em toda etapa, o valor verdadeiro está dentro do intervalo ainda vivo, e o intervalo tem metade da largura da etapa anterior. Depois de dez etapas, sua largura é de um degrau.

CONCEITO

Enunciado assim, o número que a aula chamou de resolução deixa de ser uma propriedade do conversor e passa a ser uma consequência geométrica: **a resolução é onde o intervalo para de encolher**. Meio grau não é o erro do LM35 nem do comparador — é a largura do último intervalo, e ela é 5 V dividido por dois dez vezes.

A busca binária não volta atrás. Se uma decisão sair errada, metade da faixa foi descartada para sempre, e as decisões seguintes trabalham com afunilamento dentro do intervalo errado.

ATENÇÃO

Os dois casos são muito diferentes. Errar o último bit custa um código. Errar o primeiro custa 512 — o resultado sai plausível, sem nenhum sinal de erro, e apenas com o valor pela metade. É esta assimetria que faz o mínimo de T_{AD} da próxima seção ser um limite duro e não uma recomendação: as primeiras decisões são justamente as que exigem que o conversor já tenha estabilizado.

Vale a idade do algoritmo. Em julho de 1947, no *Bell System Technical Journal*, W. M. Goodall descreveu um sistema experimental de telefonia por modulação por código de pulsos que usava um conversor por aproximações sucessivas de 5 bits a 8 mil amostras por segundo, baseado na subtração de cargas de peso binário de um capacitor.

NOTA

Mesmo ano em que o transistor foi demonstrado, e o capacitor já estava lá, fazendo o que o C_{HOLD} do 4550 faz hoje. O algoritmo é mais velho que a tecnologia inteira que o executa.

Conversores modernos deliberadamente não dividem ao meio. Como um erro de decisão é irreversível, e como os erros vêm da estabilização incompleta do conversor interno, os projetos de alto desempenho usam uma razão **menor** que dois — o espaço de busca encolhe menos da metade a cada etapa, os intervalos consecutivos se sobrepõem, e a sobreposição permite que uma etapa posterior conserte o erro de uma anterior.

Paga-se com mais etapas e ganha-se em velocidade, porque cada etapa pode ser mais curta. A referência fundadora é Kuttner (ISSCC 2002); o assunto se chama *redundância* ou *busca não binária* na literatura de circuitos.

Repare no que isso significa para o curso: a busca binária é ótima em número de comparações, e deixa de ser ótima quando as comparações são **falíveis**. O 4550 usa a versão clássica, e é por isso que ele exige que você respeite o T_{AD} .

4 O protocolo

Este é o mini datasheet da aula: o que precisa estar pronto antes de converter, como configurar o tempo, e o que acontece em cada microssegundo. **Tudo em binário**, porque em hexadecimal os campos desaparecem dentro do byte.

4.1 O que precisa estar pronto antes da primeira conversão

Sete decisões. Nenhuma delas tem um padrão razoável no reset — todas precisam ser escritas.

Decisão	Onde	Se faltar
O pino é entrada	TRISA	O microcontrolador dirige o pino e você mede a si mesmo
O pino é analógico	PCFG em ADCON1	Leitura presa em zero ou em lixo
Quais são as referências	VCFG em ADCON1	A régua fica indefinida
Quanto dura uma decisão	ADCS em ADCON2	Bits altos errados, valor plausível
Quanto dura a aquisição	ACQT em ADCON2	Leitura sempre abaixo do valor real
De que lado alinha o resultado	ADFM em ADCON2	Resultado multiplicado por 64
Qual canal, e ligar o módulo	CHS e ADON em ADCON0	Nada acontece

4.2 ADCON1 — o pino e as referências

b7	b6	b5	b4	b3	b2	b1	b0
—	—	VCFG1	VCFG0	PCFG3	PCFG2	PCFG1	PCFG0
0	0	0	0	1	1	1	0

VCFG1 escolhe a referência inferior: 0 é VSS, 1 é o pino AN2. VCFG0 escolhe a superior: 0 é VDD, 1 é o pino AN3 (RA3). Com 00, a régua vai de 0 V a 5 V, e um degrau vale 4,8828125 mV.

PCFG<3:0> é uma fronteira, não um mapa de bits: o valor diz **até qual canal** o pino é analógico. 1111 deixa tudo digital; 1110 deixa só AN0 analógico; 1101, AN0 e AN1; e assim por diante, descendo.

PCFG não tem um bit por pino. Escrever 1110 esperando “só o bit 0 ligado” é o erro clássico — e ele funciona por acidente neste caso, porque o valor certo para AN0 também é 1110. Com dois canais, o acidente acaba.

4.3 ADCON2 — o tempo

b7	b6	b5	b4	b3	b2	b1	b0
ADFM	—	ACQT2	ACQT1	ACQT0	ADCS2	ADCS1	ADCS0
1	0	0	1	0	1	0	1

4.3.1 ADCS — quanto dura uma decisão

T_{AD} é o período de uma decisão da busca binária. Ele sai do oscilador por um divisor.

Com $F_{OSC} = 16$ MHz, portanto $T_{OSC} = 62,5$ ns:

ADCS	Divisor	T_{AD}	Veredito
000	$F_{OSC}/2$	125 ns	Viola o mínimo
100	$F_{OSC}/4$	250 ns	Viola o mínimo
001	$F_{OSC}/8$	500 ns	Viola o mínimo
101	$F_{OSC}/16$	1 μ s	Escolha do curso
010	$F_{OSC}/32$	2 μ s	Válido, e mais lento
110	$F_{OSC}/64$	4 μ s	Válido, e bem mais lento
011, 111	RC interno	~ 1 a 3 μ s	Independente do clock; serve para converter dormindo

O piso é 0,7 μ s (folha de dados, tabela 28-29). Abaixo dele o comparador não estabiliza a tempo, e a busca binária erra sem avisar. **O teto é 25 μ s**, porque o capacitor não segura carga indefinidamente. Note que o reset entrega 000, que viola o piso.

4.3.2 ACQT — quanto dura a aquisição

Depois que você escreve G0, o hardware mantém a chave fechada por este tempo antes de começar a busca. Com 000, ele não espera nada — a aquisição passa a ser responsabilidade do seu programa.

000	001	010	011	100	101	110	111
0	2	4	6	8	12	16	20

Os valores são em T_{AD} . Com $T_{AD} = 1$ μ s, 010 dá 4 μ s.

Quanto é preciso? O tempo exigido não é uma constante: é uma soma que depende da impedância da fonte e da temperatura ambiente.

$$T_{ACQ} = T_{AMP} + T_C + T_{COFF}$$

Com $T_{AMP} = 0,2$ μ s fixos, $T_C = -C_{HOLD}(R_{IC} + R_{SS} + R_S) \ln(1/2048)$ e $T_{COFF} = 0,05$ μ s por grau acima de 25 °C. Para o LM35 ligado direto no pino, isso dá cerca de 1,3 μ s; para uma fonte de 500 k Ω , cerca de 96 μ s. Os 4 μ s do curso cobrem o primeiro caso com folga deliberada, e não cobrem o segundo de jeito nenhum.

4.3.3 ADFM — de que lado o resultado encosta

Com ADFM = 1, o resultado encosta à direita: ADRESL recebe os oito bits baixos e ADRESH recebe os dois altos, com os seis restantes em zero.

ADFM	ADRESH	ADRESL
1	000000 b9 b8	b7 b6 b5 b4 b3 b2 b1 b0
0	b9 b8 b7 b6 b5 b4 b3 b2	b1 b0 000000

O reset entrega 0. Um programa escrito para ler à direita, rodando com alinhamento à esquerda, recebe o valor multiplicado por 64 — e nada avisa.

4.4 ADCON0 — o canal e a partida

b7	b6	b5	b4	b3	b2	b1	b0
—	—	CHS3	CHS2	CHS1	CHS0	GO/DONE	ADON
0	0	0	0	0	0	0	1

CHS<3:0> é o número do canal em binário: 0000 é AN0, 0001 é AN1, e assim por diante. É este campo que comanda o multiplexador de entrada — só um pino de cada vez chega ao capacitor.

ADON liga o módulo. GO/DONE é um bit com dois sentidos de leitura: você escreve 1 para disparar, e o hardware devolve a 0 quando termina. Ler esse bit é ler a máquina de estados do conversor.

4.5 A linha do tempo de uma conversão

Etapas	Duração	O que está acontecendo
Repouso	—	ADON = 1, chave fechada, o capacitor segue o pino
GO → 1	—	O programa dispara
Aquisição	$ACQT = 4 T_{AD} = 4 \mu s$	Chave ainda fechada, capacitor terminando de carregar
Abertura da chave	instantânea	Aqui é a amostra. O valor congela
Decisões	$11 T_{AD} = 11 \mu s$	Dez comparações mais a entrega do resultado
GO/DONE → 0	—	ADRESH: ADRESL válidos
Espera obrigatória	2 a 3 T_{AD}	Antes de disparar a próxima

Total: 15 μs , ou 60 ciclos de máquina, com $T_{cy} = 250 \text{ ns}$. Uma atualização de tela do encontro 3 custa 5200. Ler a temperatura é a coisa mais barata que este programa faz.

A espera obrigatória entre conversões aparece como 3 T_{AD} na página 263 da folha de dados e como 2 T_{AD} na página 267. Adote a maior e registre a divergência. Para o termostato, que lê dez vezes por segundo, nenhuma das duas tem consequência.

DIVERGÊNCIA

O capacitor de retenção **não** é descarregado entre conversões: ele começa cada aquisição com a tensão da conversão anterior. Ler outro canal antes do LM35 é o caso caro — o sensor precisa puxar a carga para baixo, e ele drena no máximo 1 μA .

NESTE KIT

4.6 O número do encontro 3, pago

$$15 \mu s \div 250 \text{ ns} = \mathbf{60 \text{ ciclos de máquina}}$$

O número afirmado no encontro 3 era esse. Sessenta ciclos contra cinco mil e duzentos: uma leitura de temperatura custa menos de 1,2% do que custa mostrá-la na tela.

CONCEITO

A consequência de projeto é contraintuitiva e vale guardar: **num termostato, o gargalo não é medir, é escrever.** Quem quiser mais tempo de processador não deve amostrar menos — deve atualizar a tela menos.

5 O código, linha a linha

```
#define _XTAL_FREQ 16000000UL // antes do include: as macros de atraso dependem disso
#include <xc.h>
#include <stdint.h>

void adc_iniciar(void) {
    TRISA0bits.TRISA0 = 1; // 1. o pino é entrada
    ADCON1 = 0b00001110; // 2. AN0 analógico; referências em VDD e VSS
    ADCON2 = 0b10010101; // 3. direita | 4 TAD de aquisição | FOSC/16
    ADCON0 = 0b00000001; // 4. canal AN0, módulo ligado
}

uint16_t adc_ler(void) {
    ADCON0bits.GO = 1; // 5. dispara
    while (ADCON0bits.GO) { } // 6. espera o hardware devolver o bit a zero
```

```
return ((uint16_t)ADRESH << 8) | ADRESL; // 7. junta os dez bits
}
```

5.1 1. *TRISA0 = 1* — o pino é entrada

ADCON1 decide se o pino é analógico, mas não decide o sentido. São dois mecanismos independentes, e o conversor não corrige o outro.

Se faltar: funciona hoje, porque o reset deixa todos os pinos como entrada. Quebra no dia em que qualquer trecho anterior escrever em TRISA — e o sintoma é uma leitura constante, ou o valor que o próprio microcontrolador está impondo ao pino. É o mesmo princípio de LAT antes de TRIS: declare, não herde.

5.2 2. *ADCON1 = 0b00001110*

Os quatro bits baixos são PCFG, a fronteira do analógico: AN0 analógico, o resto digital. Os bits 5 e 4 são VCFG em 00, a régua de 0 a 5 V.

É esta linha que substitui o 0b00001111 do início da aula: primeiro tudo digital, depois o canal que se quer medir é declarado analógico.

Se faltar: o reset deixa vários pinos em modo analógico. No canal que você quer converter, a leitura vem de um pino que talvez nem esteja com a chave analógica ligada; nos outros, o buffer digital fica desligado e aquele bit de PORTx devolve zero para sempre. É o defeito que não trava nada e some quando você olha.

5.3 3. *ADCON2 = 0b10010101*

Lido em campos, da esquerda para a direita: 1 é ADFM, alinhamento à direita; 0 é o bit não implementado; 010 é ACQT, quatro T_{AD} de aquisição; 101 é ADCS, $F_{OSC}/16$, um microssegundo por decisão.

Se faltar: o reset entrega 0b00000000, que é o pior byte possível — alinhado à esquerda, sem aquisição nenhuma e com T_{AD} de 125 ns, abaixo do mínimo. A conversão acontece, o número sai, e as três coisas estão erradas em silêncio.

5.4 4. *ADCON0 = 0b00000001*

CHS em zero seleciona AN0; ADON liga o módulo. G0 fica em zero: não se dispara na inicialização.

Se faltar ADON: o G0 é escrito e nunca volta a zero. O programa trava no while do passo 6 — e este é o único defeito desta lista que se manifesta como travamento, e não como número errado.

5.5 5. *ADCON0bits.G0 = 1*

Escreve-se 1 para disparar. Use o nome qualificado: o símbolo solto G0 existe em algumas versões do XC8 e some em outras, e ADCON0bits.G0 diz de qual registrador se trata.

Um atraso de software aqui é desnecessário e prejudicial. Como ACQT não é zero, o hardware já reserva a janela de aquisição depois do disparo. Um `__delay_us` antes do G0 não acrescenta tempo de aquisição nenhum, e ainda esconde um ACQT mal configurado — o defeito passa a existir sem sintoma.

5.6 6. *while (ADCON0bits.G0) { }*

Espera cega, e desta vez legítima: quem zera o bit é o hardware, não o programa. Compare com o encontro 3, onde o display não avisava nada e a espera era por tempo fixo. Aqui existe um bit de estado, e é por isso que a espera é honesta.

São 15 microssegundos, ou 60 ciclos, com o processador parado.

Esse `while` é espera cega, do mesmo tipo que o do HD44780 — e, como lá, ele existe porque o programa não tem nada melhor a fazer **ainda**. A diferença é o preço: 60 ciclos contra 5200.

DIVERGÊNCIA

No encontro 7 essa espera vira uma interrupção, e o número deixa de importar. Mas observe a ordem em que o curso faz isso: primeiro medir o custo, depois decidir se vale eliminá-lo. Trocar `while` por interrupção antes de saber que são 60 ciclos é otimizar no escuro.

5.7 7. `((uint16_t)ADRESH << 8) | ADRESL`

O molde para `uint16_t` **antes** do deslocamento é obrigatório. Sem ele, o compilador desloca um `unsigned char` oito casas à esquerda e o resultado é zero: os dois bits altos desaparecem, e a leitura fica presa em 0 a 255.

Mascarar `ADRESH` com `0b00000011` é opcional com `ADFM = 1`, já que os seis bits altos são lidos como zero — mas declara a intenção, e protege o dia em que alguém trocar o alinhamento.

5.8 O que ainda falta

A função devolve um número de 0 a 1023, e o visor precisa de graus. O XC8 na versão gratuita não tem ponto flutuante, então a conversão é inteira:

```
uint16_t decimos = ((uint32_t)n * 625) >> 7; // 4,8828125 mV = 625/128
```

O molde para `uint32_t` é o assunto do Exercício 4.2: sem ele, o produto estoura os 16 bits no código 105, e a temperatura exibida despenca a partir de cerca de 51 °C. O defeito é invisível na bancada em dia frio.

6 O número que sai

6.1 Um degrau

A referência de fundo de escala neste kit é a própria alimentação, 5 V. Dez bits dividem essa faixa em 1024 degraus:

$$1 \text{ LSB} = 5 \text{ V} / 1024 = 4,8828125 \text{ mV}$$

Não é aproximação: 5/1024 é exato em binário. E o LM35 entrega 10 mV por grau, então:

$$1 \text{ LSB} = 0,48828125 \text{ °C}$$

Meio grau. Esta é a resolução do termostato do semestre, e ela não melhora com código melhor. Não existe leitura que distinga 24,7 de 25,0 °C nesta cadeia de medição.

CONCEITO

Guarde o número. No encontro 13 ele é o argumento que elimina o termo derivativo do controlador: derivar um sinal que só muda em degraus de meio grau produz ruído, não informação.

6.2 Noventa por cento da faixa não é usada

O fundo de escala corresponde a 5 V, ou 500 °C de LM35. O termostato opera entre a temperatura ambiente e uns 60 °C. A figura da busca binária já mostrou isso: os quatro bits mais significativos saem zero em toda leitura útil.

Estratégia	Resolução resultante	Custo
$V_{\text{REF}} = V_{\text{DD}} = 5 \text{ V}$	0,488 °C	Nenhum. É o que o kit faz
$V_{\text{REF}+}$ externo em 1 V no pino RA3	0,098 °C	Uma referência estável a mais, e perde-se um canal
Amplificar o LM35 por 5 antes do pino	0,098 °C	Um amplificador operacional e o erro de ganho dele

Nenhuma das duas alternativas entra no roteiro deste semestre — mas as duas são exatamente o que se faz em instrumentação industrial, e o aluno de Sistemas de Potência

NOTA

vai reencontrá-las em transdutores de corrente e de tensão. Aqui basta reconhecer que **resolução é uma decisão de projeto do circuito de entrada**, e não uma propriedade do microcontrolador.

6.3 Aritmética inteira

Não use ponto flutuante para isso. A conversão exata é uma fração:

$$\text{décimos de grau} = N \cdot 625/128$$

porque $4,8828125 = 625/128$, e 128 é uma potência de dois. Em C:

```
uint16_t n = ((uint16_t) ADRESH << 8) | ADRESL; // 0 a 1023
uint16_t decimos = (uint16_t)(((uint32_t) n * 625u) >> 7);
```

O produto intermediário estoura 16 bits: com $N = 1023$ ele vale 639375. Sem o `(uint32_t)` explícito, o compilador faz a conta em 16 bits e o resultado dá voltas — e dá voltas **silenciosamente**, produzindo temperaturas plausíveis e erradas.

ATENÇÃO

Este é o mesmo tipo de defeito do encontro 2: nada trava, o número simplesmente está errado. Vale colocar um `assert` mental em toda multiplicação de um valor de ADC.

6.4 O meio da gaveta

O conversor não devolve uma tensão: devolve a gaveta em que ela caiu. Código 55 significa “alguma coisa entre 268,6 mV e 273,4 mV”, e a conta acima toma a borda **inferior** dessa faixa. O resultado sai sistematicamente baixo, até meio grau, e 0,24 °C em média.

Somar metade do divisor antes de dividir corrige o viés:

```
uint16_t decimos = (uint16_t)(((uint32_t) n * 625u + 313u) >> 7);
```

O erro máximo deixa de ser meio grau só para baixo e passa a ser um quarto de grau para cada lado. Nenhum bit novo foi adquirido — o que mudou foi **qual ponto do intervalo** o programa escolhe escrever no visor. É a diferença entre resolução e viés, e ela custou uma constante.

CONCEITO

7 Ruído e a média que não ajuda

Existe um reflexo comum: amostrar quatro vezes e dividir por quatro para “ganhar resolução”. Ele funciona sob uma condição, e a condição costuma passar despercebida.

A média só melhora a resolução se houver ruído **maior que um LSB** somado ao sinal. Com ruído, amostras diferentes caem em degraus diferentes e a média recupera informação entre degraus. Sem ruído, as quatro amostras devolvem exatamente o mesmo código, e a média dele com ele mesmo é ele mesmo.

CONCEITO

Um conversor limpo demais não pode ser melhorado por média. É um resultado que parece paradoxal e não é: a quantização já jogou fora a informação, e nenhuma soma a traz de volta.

Média continua valendo para o que ela realmente faz — reduzir a variância de leitura em presença de interferência da rede, do PWM do cooler e do relé. Isso é assunto do R5 e volta no encontro 13, sob o nome de **resolução efetiva**.

8 Previsão para o R5

PREVISÃO

P1. Você vai ler o pino AN0 como entrada digital, com `PORTAbits.RA0`, antes de escrever qualquer coisa em `ADCON1` — e depois com o canal analógico habilitado. O que cada leitura devolve? Escreva os dois valores e a razão antes de ligar a placa.

P2. Com o LM35 em temperatura ambiente, qual código de dez bits você espera ler? Mostre a conta.

P3. Você vai medir o mesmo ponto vinte vezes seguidas, sem tocar em nada. Quantos códigos distintos espera ver? Justifique a partir do que foi dito sobre ruído e média.

P4. Aqueça o sensor com o dedo e o código sobe. Quantas unidades de código por grau? E qual é o menor aquecimento que a sua bancada consegue detectar?

P5. Se você trocar ACQT de $4 T_{AD}$ para 0, qual dos dois erros aparece — um deslocamento constante em todas as leituras, ou um espalhamento maior em torno do mesmo valor? A resposta depende da impedância da fonte; diga qual você espera com o LM35 e por quê.

Leve esta folha preenchida. Cada pergunta vale 0,4 — 2,0 no total do R5. Como sempre, a previsão é avaliada pelo raciocínio, e não por acertar o número.

SEM NOTA

9 Exercícios

Exercício 4.1. Um colega liga um divisor resistivo de dois resistores de $1\text{ M}\Omega$ para medir uma tensão de 0 a 10 V com o ADC. No multímetro o divisor entrega exatamente metade da tensão; no ADC, a leitura vem sempre baixa.

TAREFA

(a) O erro é de aquisição ou de conversão?

(b) Ele é sistemático ou aleatório?

(c) Dê duas correções, e diga qual delas você usaria se o consumo do circuito importasse.

Exercício 4.2. Um programa lê o LM35 e mostra a temperatura. As leituras vêm plausíveis até cerca de $33\text{ }^{\circ}\text{C}$ e, acima disso, o valor exibido despenca para perto de zero e volta a subir.

TAREFA

(a) Que código de dez bits corresponde a $33\text{ }^{\circ}\text{C}$?

(b) Qual defeito da aula produz exatamente esse comportamento?

(c) Por que o erro não aparece nas temperaturas mais baixas?

Exercício 4.3. Você precisa amostrar a temperatura a 10 Hz e atualizar o display a 10 Hz, num processador de 4 MIPS.

TAREFA

(a) Que fração do tempo de processador cada tarefa consome?

(b) Um colega propõe amostrar a 100 Hz “para ter mais dados”. Isso é caro?

(c) E atualizar o display a 100 Hz?

Exercício 4.4. Justifique, em uma frase cada, por que as três afirmações abaixo estão erradas.

TAREFA

(a) “O ADC tem 10 bits, então ele mede com precisão de 10 bits.”

(b) “Aumentar T_{AD} melhora a resolução.”

(c) “Se um bit de PORTx devolve sempre zero, o pino está com defeito.”

Exercício 4.5. Sobre a estratégia de busca do conversor.

TAREFA

(a) Um colega propõe um conversor cuja primeira pergunta é “a tensão é maior que um terço da faixa?”, e que continua sempre cortando em um terço. Quantas

comparações ele precisa no pior caso para resolver 1024 códigos? Compare com dez e diga de onde vem a diferença.

(b) Com o LM35 neste kit, as quatro primeiras decisões são conhecidas antes de a conversão começar: dão zero sempre. Isso permitiria converter em seis etapas? Se não, o que teria que mudar no circuito para que as dez etapas voltassem a informar alguma coisa?

(c) Um conversor redundante corta a faixa por 1,8 em vez de por 2. Quantas etapas ele precisa para 1024 códigos? E por que ele ainda assim pode ser mais rápido que o binário?

No encontro 5: temporizadores. Até aqui, todo intervalo de tempo deste curso foi contado em instruções — laço vazio no encontro 0, ciclos no encontro 2, esperas cegas nos encontros 3 e 4. O temporizador é a primeira peça do chip que conta tempo **sem** o processador, e ele existe exatamente para devolver os 60 e os 5200 ciclos que hoje ficam parados esperando.

NOTA